

SOC Purple Team en vivo con Wazuh

Un SOC real en AWS, atacado desde Kali, con un panel que muestra en vivo la cobertura de MITRE ATT&CK sobre alertas reales

Miguel Ángel Peralta Cano • github.com/AnyelokDev

FICHA DEL PROYECTO

Autor	Miguel Ángel Peralta Cano
Repositorio	https://github.com/AnyelokDev/soc-purple-team-wazuh
Stack	Wazuh 4.14.7, Amazon EC2 (Amazon Linux 2023), Python 3 (librería estándar), HTML/JS, Kali Linux, Fable 5.1 vía Claude Code
Contexto	Construido y presentado en el evento Fable 5.1 Build Day (septiembre de 2026)

CONTROL DOCUMENTAL

Versión	1.0	Fecha	23 de septiembre de 2026
Estado	Terminado	Clasificación	Público • portafolio

Resumen

Monté un Centro de Operaciones de Seguridad (SOC) pequeño pero real: Wazuh 4.14.7 en una instancia de Amazon Amazon Elastic Compute Cloud (EC2), atacado desde una máquina Kali, y un panel web que lee las alertas de Wazuh en vivo y pinta sobre ellas una cuadrícula de MITRE ATT&CK con un porcentaje de cobertura. La idea era mostrar en una demo de pocos minutos el ciclo completo de un equipo *purple*: se ataca, se mira qué detecta el SOC, se identifica lo que falta y se cierra con una regla nueva.

El set de la demo son siete técnicas. Cinco las cubrieron las reglas de fábrica de Wazuh en cuanto hubo actividad. Las otras dos corresponden al **escaneo de red con nmap** (T1595 y T1046). El escaneo funcionó: llegó al servidor, identificó los servicios expuestos y dejó sus huellas en el log de SSH. Lo que no existía era una regla que interpretara esos eventos, así que para el Wazuh de fábrica el escaneo era invisible. Con un decoder y tres reglas de correlación nuevas, el escaneo pasó a detectarse y el panel subió del 43 % al 71 %, hasta llegar al **100 % de cobertura con alertas reales**.

El aporte principal del proyecto es cómo se escribieron esas reglas. Yo diseñé el flujo, monté el SOC, lancé los ataques y dirigí el trabajo. Dentro de ese flujo, Fable 5.1 (vía Claude Code) generó de forma autónoma el decoder y las reglas de detección, y las fue mejorando a partir de lo que el Wazuh real registraba hasta que el escaneo quedó detectado. También escribió la regla de persistencia (T1136) y el panel. Todo se validó contra alertas reales. Durante la demo el panel no consulta al modelo; todo lo que muestra sale de `alerts.json`.

Queda pendiente un modo de anonimización en el panel, para poder grabarlo sin que aparezcan IPs ni nombres de host reales.

Qué demuestra este proyecto

Competencia	Dónde se ve
Desplegar y endurecer un SIEM en la nube	Wazuh todo en uno en EC2, SSH solo con clave (sección 2).
Detección como código con un modelo de IA	Fable 5.1 generó de forma autónoma el decoder <code>sshd-banner-invalid</code> y las reglas 100098–100100 dentro del flujo que diseñé (sección 3.1).
Mapeo a MITRE ATT&CK	Matriz de cobertura de las siete técnicas (Anexo A).
Análisis con evidencia	El caso T1136, donde la evidencia contradujo la hipótesis inicial (sección 6).
Desarrollo de herramientas propias	Panel en vivo con Python y SSE, sin dependencias (sección 3.2).
OPSEC al publicar	Capturas censuradas y revisadas; tres fugas que el OCR no vio (sección 7).

Tabla 1. Competencias y dónde se respaldan.

Resultados clave

Resultado 1 — 100 % de cobertura del set con alertas reales

Las siete técnicas del set (T1595, T1046, T1110, T1078, T1021, T1548 y T1136) quedaron cubiertas por alertas reales de Wazuh.

Evidencia: capturas 14, 18 y 22 ([Anexo D](#)).

Resultado 2 — el escaneo pasó de invisible para Wazuh a detectado

El escaneo con nmap se completó contra el servidor (captura 05), pero el Wazuh de fábrica no tenía una regla que interpretara esos eventos, así que no generaba alerta. Con el decoder y la regla de correlación 100100, escritos por Fable, el mismo escaneo produce una alerta de nivel 8 mapeada a T1046 y T1595, y la cobertura del panel sube del 43 % al 71 %.

Evidencia: capturas 05 a 08.

Resultado 3 — publicación sin datos reales

El repositorio no contiene IPs, nombres de host ni credenciales reales. Las capturas usan direcciones de documentación (RFC 5737), y en la revisión final aparecieron tres fugas que la verificación con OCR había pasado por alto.

Evidencia: [sección 7](#).

Índice

Resumen	1
Qué demuestra este proyecto	1
Resultados clave	2
1 Contexto y objetivo	4
1.1 Reparto del trabajo	4
2 Arquitectura	4
3 Implementación	5
3.1 Reglas y decoder propios	5
3.2 Panel en vivo	6
3.3 Scripts de apoyo	6
4 La demo paso a paso	6
5 Decisiones de diseño	8
6 Problemas encontrados	9
7 Seguridad del propio proyecto	10
8 Limitaciones y trabajo futuro	11
9 Conclusiones	12
Anexo A Matriz de cobertura ATT&CK	13
Anexo B Reglas y decoder	13
Anexo C Montaje desde cero	14
Anexo D Evidencia	15
Glosario de siglas	18
Referencias	18

1 Contexto y objetivo

El proyecto nació para el *Fable 5.1 Build Day*, un evento en el que había que construir algo con el modelo Fable 5.1 y presentarlo en vivo. Elegí un tema de mi área: automatizar parte del trabajo de un analista de SOC, que es revisar qué técnicas de ataque detecta el gestión de información y eventos de seguridad (SIEM), cuáles no, y escribir las reglas que faltan.

El objetivo concreto fue una demo en la que el público viera:

- 1. Ataques reales contra un servidor real, no un video ni datos simulados.
- 2. Cada alerta de Wazuh apareciendo en pantalla en el momento en que ocurre, con su técnica de MITRE ATT&CK (The MITRE Corporation, [s.f.](#)).
- 3. Un porcentaje de cobertura sobre un set cerrado de técnicas, que sube a medida que se detectan.
- 4. Un hueco de detección y su cierre con una regla escrita para el caso.

1.1 Reparto del trabajo

Como el proyecto se hizo con un modelo de IA, dejo claro qué hizo cada parte:

Quién	Qué hizo
Yo	Montaje de la infraestructura (EC2, Wazuh, endurecimiento de SSH), definición del set de técnicas y de la demo, dirección y validación del trabajo del modelo, ejecución de los ataques, revisión OPSEC y presentación.
Fable 5.1 (vía Claude Code)	Escribió el panel (<code>server.py</code> e <code>index.html</code>), el decoder y las reglas de escaneo, y la regla 100200, que detecta la creación de cuentas locales, la mapea a T1136 (persistencia) y la eleva a nivel 10.

Tabla 2. Reparto del trabajo.

2 Arquitectura

Todo el lado defensivo vive en una sola instancia EC2. Wazuh corre en modo todo en uno (manager, indexer y dashboard) y vigila el propio servidor: los eventos de SSH, PAM y del sistema del host son la telemetría. El panel en vivo corre en la misma máquina y solo lee el archivo de alertas del manager.

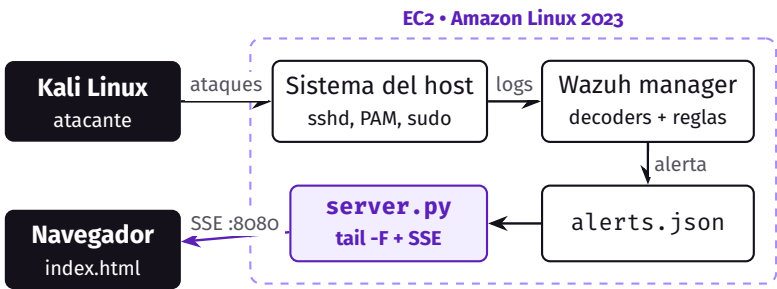


Figura 1. Arquitectura del laboratorio.

Componente	Detalle
Servidor	EC2 t3.large con Amazon Linux 2023 e IP fija (Elastic IP). SSH solo con clave; usuario ec2-user.
SIEM	Wazuh 4.14.7 todo en uno, instalado con el asistente oficial (Wazuh Inc., s.f.-d).
Panel	dashboard/server.py (Python, solo librería estándar) e index.html. Escucha en el puerto 8080.
Reglas propias	wazuh-rules/: un decoder, tres reglas de escaneo y la regla 100200.
Atacante	Kali Linux en VirtualBox. El escaneo y los intentos de acceso salen de ahí; la creación de cuentas se hizo en la propia EC2.

Tabla 3. Componentes.

3 Implementación

3.1 Reglas y decoder propios

Wazuh ya trae reglas para casi todo el set. Lo que faltaba era una forma de detectar el escaneo de red, que sí llegaba al servidor. El problema tenía dos partes (sección 6): la regla de fábrica para «conexión cerrada» tiene nivel 0, así que no genera alerta, y las sondas no-SSH de nmap -A quedan en el log de OpenSSH 9.x como banner exchange: ... invalid format, un mensaje que ningún decoder de fábrica interpreta, así que tampoco se extrae la IP de origen.

La solución tiene tres piezas, siguiendo la sintaxis de reglas de Wazuh (Wazuh Inc., s.f.-e):

ID	Nivel	Qué hace	MITRE
decoder	—	sshd-banner-invalid: extrae srcip y srcport del mensaje banner exchange.	—
100099	1	Hija de la 5722 («conexión cerrada»). La sube a nivel 1 para que cuente como evento base.	—
100098	1	Sonda con protocolo inválido en el puerto SSH. Evento base.	—
100100	8	Dos o más eventos base desde la misma IP en 5 minutos: escaneo.	T1046 T1595
100200	10	Hija de la 5902 («cuenta creada»): detecta la creación de cuentas locales como persistencia, la mapea a T1136 y la sube de nivel 8 a 10.	T1136

Tabla 4. Decoder y reglas propias.

Las otras técnicas las cubre Wazuh de fábrica (Wazuh Inc., s.f.-b): 5710 y 5712 (usuario inexistente y fuerza bruta; T1110 y T1021), 5715 y 5501 (acceso válido; T1078 y T1021), 5402 (sudo a root; T1548.003) y 5902 (cuenta creada; T1136, persistencia).

3.2 Panel en vivo

`server.py` lanza un `tail -F` sobre `/var/ossec/logs/alerts/alerts.json`, convierte cada línea en un objeto reducido (regla, nivel, técnicas, IP de origen, usuario) y lo envía a los navegadores conectados por Server-Sent Events (SSE) (WHATWG, [s.f.](#)). Además:

- Guarda las últimas 300 alertas y las reenvía cuando un navegador se conecta, para que el panel no arranque vacío si se recarga.
- Manda un latido cada 15 segundos y le indica al navegador que se reconecte a los 2 segundos si se cae la conexión.
- Expone `/health` y `/api/recent` para comprobar el estado sin abrir el panel.

`index.html` mantiene el estado de la demo: el flujo de cuatro etapas, la cuadrícula de las siete técnicas y el porcentaje. Una casilla pasa a «cerrado» cuando llega una alerta real mapeada a esa técnica. El botón «Reiniciar demo» pone todo a cero y guarda ese punto en el navegador.

Sobre la etapa «Fable 5.1» del panel

El flujo del panel muestra cuatro etapas: Kali, Wazuh, Fable 5.1 y Regla. La tercera es solo visual: repite el mapeo MITRE que ya trae la alerta de Wazuh. El panel no llama a ningún modelo en tiempo real. Fable participó en la construcción, no en la ejecución.

3.3 Scripts de apoyo

- `dashboard/run.sh`: arranca, detiene y revisa el panel (`start`, `stop`, `restart`, `status`, `logs`).
- `scripts/close_gap_t1136.sh`: hace copia de seguridad de las reglas, añade la 100200 si no está y reinicia el manager.
- `scripts/enable_active_response.sh`: activa el bloqueo automático (`firewall-drop`) para alertas de nivel 10 o más (Wazuh Inc., [s.f.-c](#)). Trae marcadores para poner las IPs de gestión en la lista blanca antes de ejecutarlo.

4 La demo paso a paso

La demo empieza con el panel en cero y avanza por fases. La [tabla 5](#) resume cómo subió la cobertura según las capturas.

Fase	Capturas	Cobertura	Técnicas nuevas y regla
Acceso válido y sudo	02–04	43 %	T1078, T1021 (5715, 5501) y T1548, escalada de privilegios (5402)
Escaneo desde Kali	05–08	71 %	T1595 y T1046 (100100, propia)
Intentos de acceso desde Kali	09–13	86 %	T1110 (5710 y 5712)
Creación de cuenta en la EC2	14–18	100 %	T1136, persistencia (5902, de fábrica)
Regla 100200	20–22	100 %	T1136 detectada por la 100200 con nivel 10 aplicada

Tabla 5. Progresión de la cobertura durante la demo.

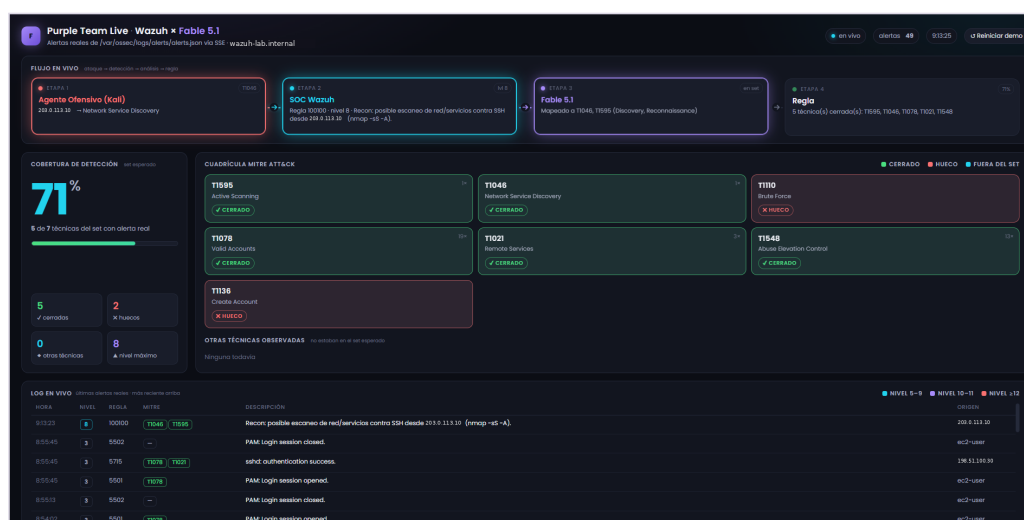


Figura 2. El escaneo de Kali detectado por la regla 100100: T1595 y T1046 pasan a «cerrado» y la cobertura sube al 71 %.

Los intentos de acceso por contraseña con una herramienta de fuerza bruta no llegaron a probarse: el servidor rechaza la autenticación por contraseña. Por eso esa fase se hizo con intentos de acceso de usuarios inexistentes, que Wazuh registra con la regla 5710 y, al repetirse, con la 5712 (sección 6).

La última fase corresponde a la persistencia mediante creación de cuentas (T1136). La secuencia fue:

1. Ejecuté `sudo useradd` en la EC2 para generar el evento de creación de cuenta (capturas 15 y 16).
2. Apliqué la regla 100200, escrita por Fable, con `close_gap_t1136.sh` (captura 21).
3. Al crear de nuevo una cuenta, la regla 100200 detectó el evento, lo mapeó a T1136 y lo elevó a nivel 10. En el log del panel aparece así: «Persistencia: cuenta local creada (mapeada a ATT&CK T1136 por Fable 5.1)» (captura 22).

El cierre del hueco consiste en eso: la creación de cuentas queda detectada y etiquetada como la técnica de persistencia que es, con un nivel que la hace visible frente al resto de eventos del sistema.

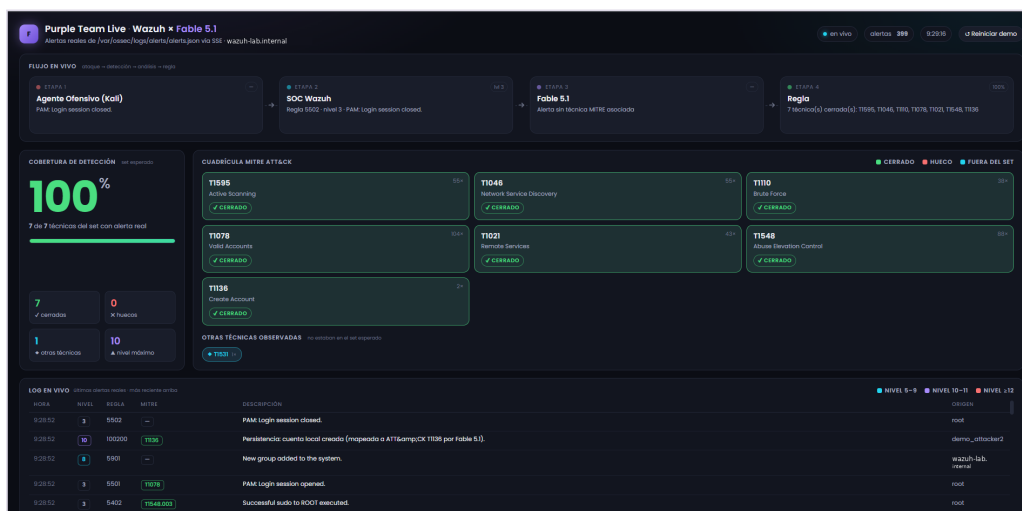


Figura 3. Panel al 100 % con la regla 100200 activa: la creación de demo_attacker2 entra como nivel 10, mapeada a T1136 (persistencia).

5 Decisiones de diseño

Decisión 1 — Python sin dependencias y SSE para el panel

Contexto. El panel tenía que mostrar alertas en el momento en que ocurren y arrancar en la misma EC2 sin instalar nada.

Qué se eligió. Un servidor con solo la librería estándar de Python y SSE hacia el navegador.

Por qué. El flujo va en un solo sentido (servidor a navegador), y SSE resuelve eso sobre HTTP normal, con reconexión automática del navegador. Sin dependencias, el panel se copia y se ejecuta en cualquier servidor con Python 3.

Qué se descartó. WebSockets (bidireccional, no hacía falta) y consultar la API del indexer cada pocos segundos (añade retraso y credenciales).

Decisión 2 — leer alerts.json con sudo -n tail -F

Contexto. El archivo de alertas es de Wazuh y el panel no debía correr como root.

Qué se eligió. Lanzar tail -F con sudo -n desde un usuario normal.

Por qué. tail -F sigue al archivo aunque Wazuh lo rote cada día, y sudo -n falla de inmediato en vez de quedarse esperando una contraseña. Solo ese comando necesita privilegios, no todo el servidor web.

Qué se descartó. Ejecutar server.py como root.

Decisión 3 — el manager se vigila a sí mismo

Contexto. Hacía falta telemetría del servidor atacado.

Qué se eligió. Usar los logs del propio host del manager, sin agente.

Por qué. El agente y el manager de Wazuh no se instalan en el mismo equipo, y el manager ya analiza sus propios logs. Para una demo de un solo servidor era suficiente.

Qué se descartó. Una segunda instancia con agente, que duplica costo y complejidad.

Decisión 4 — correlación en vez de una regla por evento

Contexto. Un solo evento de conexión cerrada no significa escaneo; pasa todo el tiempo.

Qué se eligió. Eventos base de nivel 1 y una regla de correlación (100100) que exige dos o más desde la misma IP en 5 minutos.

Por qué. Así cada conexión suelta no genera ruido, pero un escaneo sí se alerta, y con la IP del escáner.

Qué se descartó. Alertar cada conexión cerrada, que llenaría el panel de falsos positivos.

Decisión 5 — la regla 100200 en un archivo aparte

Contexto. La demo necesitaba un momento visible de «regla nueva aplicada».

Qué se eligió. Dejar la 100200 fuera de `local_rules.xml` y aplicarla en vivo con `close_gap_t1136.sh`.

Por qué. Así se puede mostrar el antes y el después sin editar archivos en el escenario. El script hace copia de seguridad y no duplica la regla si ya está.

Qué se descartó. Editar las reglas a mano durante la presentación.

6 Problemas encontrados

Problema 1 — Wazuh de fábrica no detectaba el escaneo

Síntoma. El escaneo desde Kali funcionó: llegó al servidor, encontró los puertos 22 y 443 abiertos e identificó los servicios (captura 05). Aun así, el panel no mostraba ninguna alerta para T1595 ni T1046.

Hipótesis. Los eventos del escaneo sí llegaban a Wazuh, pero ninguna regla de fábrica los interpretaba como escaneo.

Verificación. La regla de fábrica para conexión cerrada (5722) tiene nivel 0. Las sondas no-SSH aparecían en el log como `banner exchange ... invalid format`, un formato que ningún decoder extraía.

Resultado. Fable generó el decoder `sshd-banner-invalid` y las reglas 100098, 100099 y 100100, y las ajustó contra los eventos reales del servidor. El escaneo, que antes era invisible para Wazuh, ahora produce una alerta de nivel 8 con T1046 y T1595, y la cobertura sube del 43 % al 71 % (capturas 06 a 08).

Problema 2 — la fuerza bruta por contraseña no era posible

Síntoma. La herramienta de fuerza bruta respondía que el servidor no admite autenticación por contraseña (captura 10).

Hipótesis. Es el endurecimiento del propio servidor: SSH solo con clave.

Verificación. El mensaje de error lo confirma. Relajar esa configuración para la demo no tenía sentido.

Resultado. La fase se hizo con intentos de acceso de usuarios inexistentes, que Wazuh sí registra (5710) y correlaciona como fuerza bruta (5712, T1110).

Problema 3 — T1136: la hipótesis inicial era incorrecta

Síntoma. Antes de crear cualquier cuenta, la casilla T1136 estaba en rojo.

Hipótesis. Se asumió que Wazuh detectaba la creación de cuentas (regla 5902) pero no la mapeaba a MITRE, y con esa idea se escribió la regla 100200 para «cerrar el hueco».

Verificación. En la captura 14, la alerta de la 5902 ya trae la etiqueta T1136 y la casilla pasa a verde antes de aplicar la 100200. El código fuente de Wazuh 4.14 lo confirma: la regla 5902 incluye `<mitre><id>T1136</id></mitre>` (Wazuh Inc., [s.f.-a](#)). La casilla estaba en rojo porque aún no se había creado ninguna cuenta, no porque faltara el mapeo.

Resultado. La 100200 sigue siendo útil, pero por otra razón: con ella activa, la creación de la cuenta queda registrada como «Persistencia: cuenta local creada (mapeada a ATT&CK T1136 por Fable 5.1)», con nivel 10 en vez de 8, lo que la pone dentro del umbral de respuesta activa (captura 22). La lección es que un «hueco» en la cuadrícula significa «sin alerta todavía», y eso hay que comprobarlo contra el ruleset antes de escribir una regla.

Problema 4 — fugas en capturas ya censuradas

Síntoma. Las capturas se censuraron con un script (bloque opaco y reemplazo por direcciones RFC 5737), y una verificación con OCR dio cero coincidencias.

Hipótesis. El OCR no ve todo.

Verificación. Revisadas a ojo y con el contraste subido, tres capturas todavía mostraban datos reales: una terminal transparente dejaba ver detrás un comando anterior con la IP del servidor, y en otras dos la última fila del log, cortada en el borde, mostraba parte de una IP y de un nombre de host internos.

Resultado. Las copias del repositorio se recortaron y se revisaron de nuevo. La lección quedó en el checklist OPSEC de mi plantilla.

7 Seguridad del propio proyecto

El panel no tiene autenticación

`server.py` escucha en `0.0.0.0:8080` y muestra alertas reales, incluidas IPs de origen y usuarios. El grupo de seguridad de la EC2 debe permitir el puerto 8080 solo desde la IP de quien presenta.

El bloqueo automático puede bloquear al administrador

`enable_active_response.sh` bloquea por firewall las IPs que disparan alertas de nivel 10 o más. Antes de ejecutarlo hay que reemplazar los marcadores `IP_GESTION_SSH` e `IP_GESTION_NAVEGADOR`, o se pierde el acceso al servidor.

Para publicar el proyecto:

- En el código y la documentación, la IP del servidor aparece como `TU_IP_EC2`. No hay claves, contraseñas ni IPs de gestión.
- Las capturas usan direcciones de documentación (Arkko et al., 2010): `198.51.100.20` para el servidor, `203.0.113.10` para el atacante y `198.51.100.30` para el origen de las sesiones de administración. El nombre del host aparece como `wazuh-lab`.
- El `.gitignore` excluye claves, archivos `.env`, credenciales, logs de alertas y notas internas de OPSEC.

Además, la instancia sigue generando costo apagada (disco EBS e IP pública), así que al terminar conviene liberar la IP y terminar la instancia.

8 Limitaciones y trabajo futuro

El modelo no participa en la ejecución

La etapa «Fable 5.1» del panel es visual. Un siguiente paso real sería que el modelo recibiera las alertas sin técnica y propusiera la regla en vivo, con aprobación humana antes de aplicarla.

Alcance acotado

Un solo servidor, siete técnicas y ataques controlados. La cobertura mide «técnica con al menos una alerta real», no la calidad de la detección ante variantes del mismo ataque. Además, la `t3.large` (2 vCPU) está por debajo de los 4 vCPU que Wazuh recomienda para el modo todo en uno (Wazuh Inc., s.f.-d). Alcanzó para la demo, pero no sirve como referencia de rendimiento.

Modo de anonimización en el panel

Añadir en `server.py`, donde se arma cada alerta, un modo activado con `ANON=1` que reemplace IPs y nombres de host antes de enviarlos por SSE. Así se puede grabar o proyectar el panel sin censurar después.

9 Conclusiones

1. **Un modelo de IA puede escribir y mejorar reglas de detección de forma autónoma.** Es el aporte principal del proyecto. Dentro del flujo que diseñé (SOC montado, ataques definidos y validación contra alertas reales), Fable 5.1 generó el decoder y las reglas que convirtieron un escaneo invisible para Wazuh en una alerta mapeada a MITRE, y las ajustó hasta que funcionaron sobre los eventos reales. Mi papel fue diseñar el flujo, dirigir y verificar; el de Fable, producir la detección.
2. **El hueco real estaba en la detección del escaneo.** Las reglas de fábrica cubren bien los accesos, la escalada de privilegios con sudo (T1548) y la persistencia por creación de cuentas (T1136), pero no correlacionan un escaneo de red contra OpenSSH 9.x. Ahí fue donde la detección como código aportó.
3. **Un panel vacío no es un hueco.** El caso T1136 mostró que hay que verificar contra el ruleset antes de dar por hecho que algo no se detecta.
4. **La autonomía funciona mejor con un flujo que la verifique.** Que cada regla se probara contra un Wazuh real, y que la evidencia se revisara hasta encontrar el error de la hipótesis sobre T1136, fue lo que hizo confiable el resultado.
5. **Publicar también es parte del proyecto.** La revisión OPSEC encontró fugas después de una verificación automática que decía que no las había.

Anexo A. Matriz de cobertura ATT&CK

Técnica	Táctica	Actividad en la demo	Regla	Captura	Estado
T1595 Active Scanning	Reconocimiento	Escaneo desde Kali	100100	06	HECHO
T1046 Network Service Discovery	Descubrimiento	Escaneo desde Kali	100100	06	HECHO
T1110 Brute Force	Acceso a credenciales	Accesos con usuarios inexistentes desde Kali	5710, 5712	12, 13	HECHO
T1078 Valid Accounts	Acceso inicial	Acceso SSH válido	5715, 5501	03, 04	HECHO
T1021 Remote Services	Movimiento lateral	Acceso SSH	5715, 5710	03, 12	HECHO
T1548 Abuse Elevation Control	Escalada de privilegios	sudo a root	5402	04	HECHO
T1136 Create Account	Persistencia	Creación de cuenta local con useradd en la EC2	5902, 100200	14, 22	HECHO

Tabla 6. Cobertura de las siete técnicas del set. La táctica es la principal en el contexto de la demo; T1078 figura en ATT&CK bajo varias tácticas.

Anexo B. Reglas y decoder

Decoder (wazuh-rules/local_decoder.xml):

```
<decoder name="sshd-banner-invalid">
  <parent>sshd</parent>
  <prematch>banner exchange: Connection from </prematch>
  <regex offset="after_prematch">^(\S+) port (\d+):</regex>
  <order>srcip, srcport</order>
</decoder>
```

Reglas de escaneo (wazuh-rules/local_rules.xml):

```
<rule id="100099" level="1">
  <if_sid>5722</if_sid>
  <description>sshd: conexion cerrada sin autenticar (evento base).</description>
  <group>recon_base,</group>
</rule>
<rule id="100098" level="1">
  <if_sid>5700</if_sid>
  <match>banner exchange: Connection from</match>
  <description>sshd: sonda con protocolo invalido (evento base).</description>
  <group>recon_base,</group>
</rule>
<rule id="100100" level="8" frequency="2" timeframe="300">
  <if_matched_group>recon_base</if_matched_group>
  <same_srcip />
```

```
<description>Recon: posible escaneo contra SSH desde $(srcip).</description>
<mitre><id>T1046</id><id>T1595</id></mitre>
<group>recon,network_scan,</group>
</rule>
```

Regla de creación de cuentas (wazuh-rules/local_rules_t1136.xml):

```
<rule id="100200" level="10">
  <if_sid>5902</if_sid>
  <description>Persistencia: cuenta local creada (mapeada a ATT&CK T1136 por Fable 5.1).
  </description>
  <mitre><id>T1136</id></mitre>
  <group>account_manipulation,persistence,</group>
</rule>
```

Las descripciones están abreviadas; el texto completo está en los archivos del repositorio.

Anexo C. Montaje desde cero

1. Instancia. EC2 con Amazon Linux 2023 e IP fija. En el grupo de seguridad, abrir 22 y 443 solo desde tu IP, y 8080 solo desde la IP del navegador del panel.

2. Wazuh todo en uno (Wazuh Inc., [s.f.-d](#)):

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
sudo bash ./wazuh-install.sh -a
```

3. Reglas y decoder propios:

```
sudo cp wazuh-rules/local_rules.xml /var/ossec/etc/rules/local_rules.xml
sudo cp wazuh-rules/local_decoder.xml /var/ossec/etc/decoders/local_decoder.xml
sudo chown wazuh:wazuh /var/ossec/etc/rules/local_rules.xml \
    /var/ossec/etc/decoders/local_decoder.xml
sudo systemctl restart wazuh-manager
```

4. Panel. El usuario que lo arranca debe poder ejecutar `sudo -n tail` sobre `alerts.json`.

```
cd dashboard
./run.sh start      # http://TU_IP_EC2:8080
./run.sh status     # consulta /health
```

5. Regla 100200 (opcional, en vivo): `./scripts/close_gap_t1136.sh`.

Anexo D. Evidencia

Las 22 capturas están en evidencia/, numeradas en el orden en que ocurrieron. Aquí van las que sostienen los resultados, en el orden de la demo; todas usan direcciones de documentación.

Punto de partida (43 %). El panel ya muestra cerradas T1078, T1021 y T1548. Las produjeron un acceso SSH válido y un sudo a root, y las detectaron reglas de fábrica (5715, 5501 y 5402). Es la base que Wazuh cubre solo, antes de cualquier regla nueva.

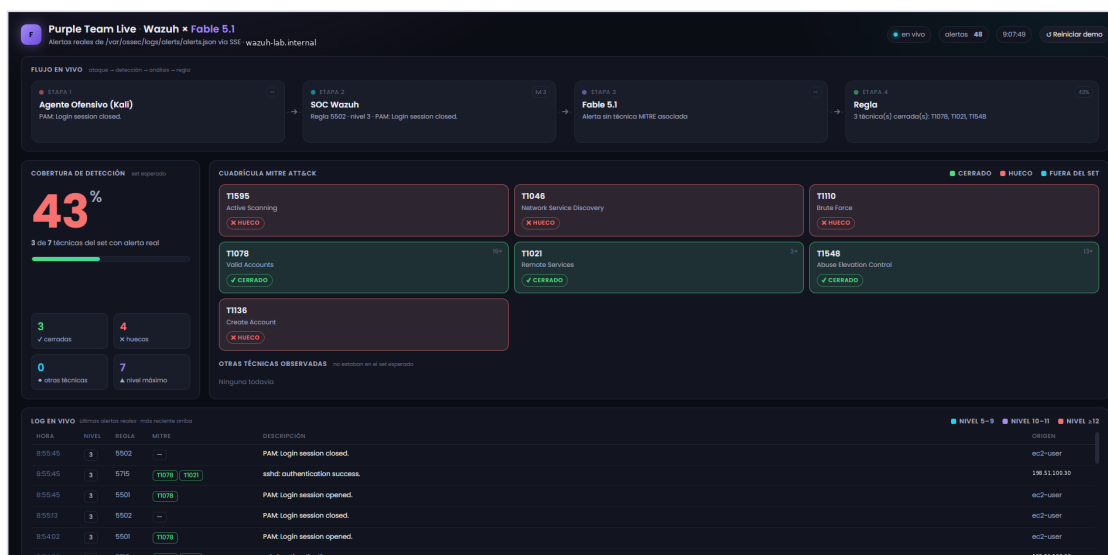


Figura 4. Captura 03. Primeras técnicas cerradas (43 %) con accesos válidos y sudo.

El escaneo llega al servidor. Desde Kali se lanzó nmap contra la EC2: encontró los puertos 22 y 443 abiertos e identificó OpenSSH 9.9 y el servicio web de Wazuh. El ataque funcionó; lo que faltaba era que Wazuh lo detectara, y eso es lo que resuelven las reglas de Fable (Resultado 2).

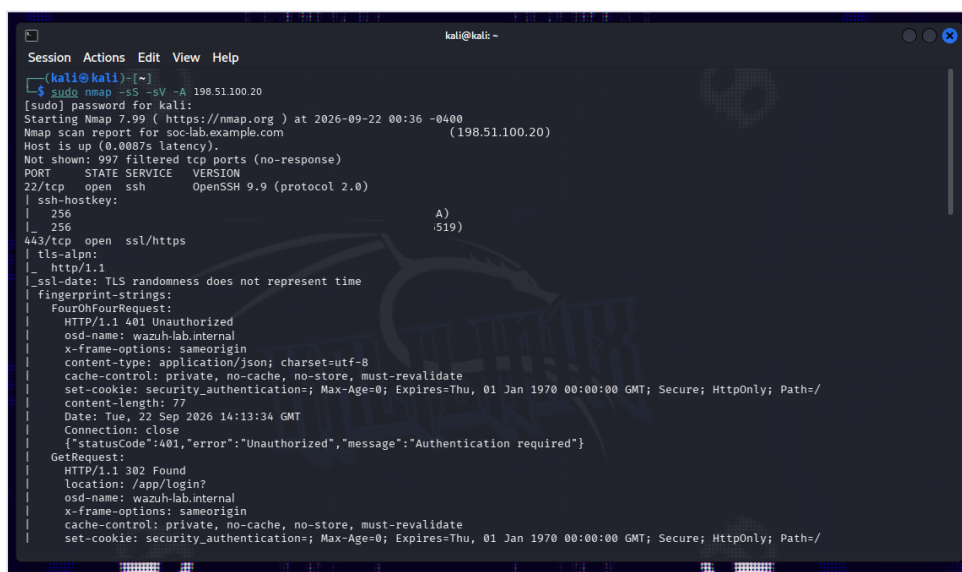


Figura 5. Captura 05. Escaneo desde Kali contra el servidor (huellas SSH tapadas).

El endurecimiento hace su trabajo. El intento de fuerza bruta por contraseña termina con el servidor respondiendo que no admite ese método de autenticación. Por eso la fase de fuerza bruta se hizo con usuarios inexistentes (Problema 2).

```
(kali@kali)~$ sudo chmod +r /usr/share/wordlists/rockyou.txt
(kali@kali)~$ hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://198.51.100.20 -t 4 -f
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-22 00:42:41
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:1/p:14344399), ~3586100 tries per task
[DATA] attacking ssh://198.51.100.20 :22/
[ERROR] target ssh://198.51.100.20 :22/ does not support password authentication (method reply 100).
```

Figura 6. Captura 10. El servidor rechaza la autenticación por contraseña.

Fuerza bruta detectada (86 %). Los intentos de acceso con usuarios inexistentes desde Kali aparecen en el log como regla 5710, mapeados a T1110 y T1021, y T1110 pasa a cerrada. Para entonces el escaneo ya figura como detectado por la regla 100100.

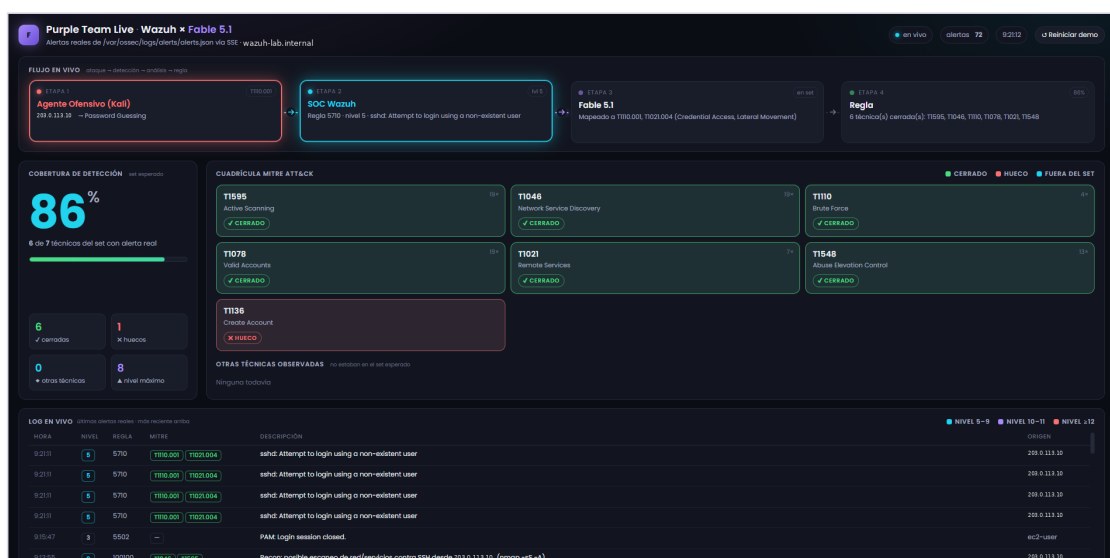


Figura 7. Captura 12. Intentos con usuarios inexistentes detectados por la 5710 (86 %).

Persistencia por creación de cuenta (100 %). El `sudo useradd` ejecutado en la EC2 genera la alerta 5902 con T1136, y el panel llega al 100 %. Esta captura es la evidencia del Problema 3: la regla de fábrica ya etiquetaba T1136 antes de aplicar la 100200.

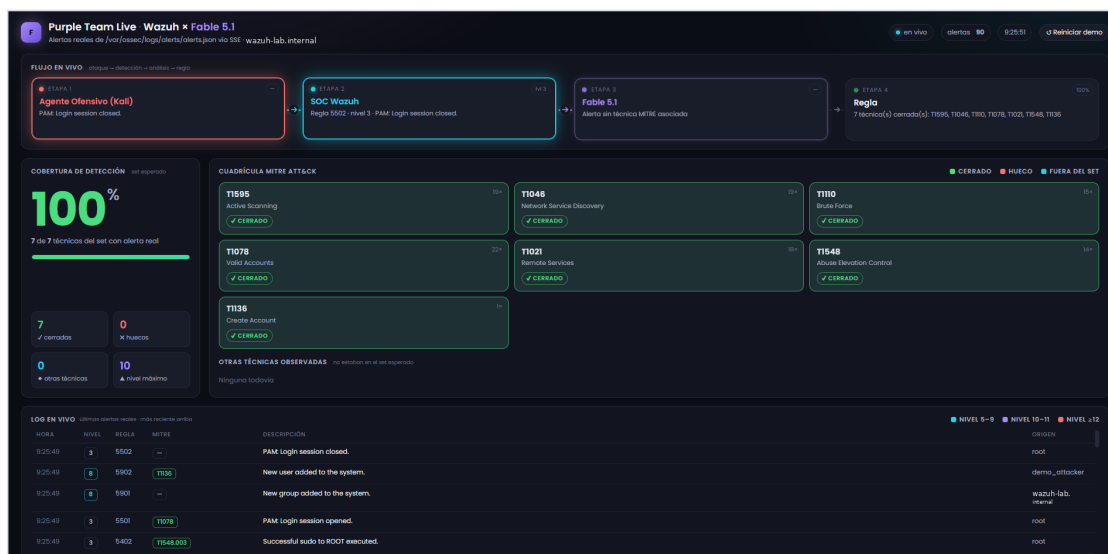


Figura 8. Captura 14. La regla de fábrica 5902 ya trae T1136: la casilla se cierra antes de aplicar la 100200.

Despliegue de la regla de Fable. En la EC2 se ejecuta `close_gap_t1136.sh`, que agrega la regla 100200 y reinicia el manager. A partir de aquí, cada cuenta nueva se registra como «Persistencia: cuenta local creada (mapeada a ATT&CK T1136 por Fable 5.1)» con nivel 10, como se ve en la captura 22 (figura 3).

```

Last login: Tue Sep 22 13:54:02 2026 from 198.51.100.30
[ec2-user@wazuh-lab ~]$ sudo useradd demo_attacker
[ec2-user@wazuh-lab ~]$ /home/ec2-user/demo/close_gap_t1136.sh
wazuh-analysisd is running...
Hueco T1136 cerrado. Repite 'sudo useradd ...' y la casilla pasa a verde.
[ec2-user@wazuh-lab ~]$

```

Figura 9. Captura 21. Aplicación de la regla 100200 con `close_gap_t1136.sh`.

Glosario de siglas

EC2 Amazon Elastic Compute Cloud

SIEM gestión de información y eventos de seguridad

SOC Centro de Operaciones de Seguridad

SSE Server-Sent Events

Referencias

Arkko, J., Cotton, M., & Vegoda, L. (2010, enero). *IPv4 Address Blocks Reserved for Documentation* (RFC N.º 5737). Internet Engineering Task Force. <https://doi.org/10.17487/RFC5737>

The MITRE Corporation. (s.f.). *MITRE ATT&CK Enterprise Matrix*. Consultado el 23 de septiembre de 2026, desde <https://attack.mitre.org/matrices/enterprise/>

Wazuh Inc. (s.f.-a). *0020-syslog_rules.xml (versión 4.14.0)*. Consultado el 23 de septiembre de 2026, desde https://github.com/wazuh/wazuh/blob/v4.14.0/ruleset/rules/0020-syslog_rules.xml

Wazuh Inc. (s.f.-b). *0095-sshd_rules.xml (versión 4.14.0)*. Consultado el 23 de septiembre de 2026, desde https://github.com/wazuh/wazuh/blob/v4.14.0/ruleset/rules/0095-sshd_rules.xml

Wazuh Inc. (s.f.-c). *Active response: Configuration*. Consultado el 23 de septiembre de 2026, desde <https://documentation.wazuh.com/current/user-manual/capabilities/active-response/remediation-configuration.html>

Wazuh Inc. (s.f.-d). *Quickstart*. Consultado el 23 de septiembre de 2026, desde <https://documentation.wazuh.com/current/quickstart.html>

Wazuh Inc. (s.f.-e). *Rules syntax*. Consultado el 23 de septiembre de 2026, desde <https://documentation.wazuh.com/current/user-manual/ruleset/ruleset-xml-syntax/rules.html>

WHATWG. (s.f.). *HTML Living Standard: Server-sent events*. Consultado el 23 de septiembre de 2026, desde <https://html.spec.whatwg.org/multipage/server-sent-events.html>